pre-processed this dataset by merging titles with bodies, removing *"tldr"* tags, removing posts that are marked as *"nsfw"* or *"+18"*, removing duplicates, and removing posts longer than 200 tokens or shorter than 20 tokens. The final cleaned dataset consists of 1458003 posts.

B.3 LLM-as-a-judge validation

Quality We measured text quality using LLM-as-a-judge method whose performance been empirically confirmed in previous studies (Chen et al., 2023). We use LLama-3.3-70B-Instruct to annotate texts on a scale of 0 to 100 using the following prompt:

Quality evaluation prompt

```
On a scale of 0 to 100, evaluate the post. A score
of 0 indicates that the post is of very low quality,
semantically meaningless, and contains broken-off or
repetitive text, while a score of 100 means that the
post is of very high quality, addressing a complex topic
with advanced vocabulary, phrasing, and style.
Post: <text>
Reply ONLY with the integer score (0-100). DO NOT reply
with text.
```

To confirm the correctness of our judge, we compare it to the judge from (Chen et al., 2023), which was shown to outperform many other quality metrics. We use the same experimental setup as in Chen et al. (2023) and compare to numbers extracted from Table 10 in Chen et al. (2023). Table 2 shows the correlations for the two judges with human annotations of the OpenMEVA-ROC dataset (Guan et al., 2021) of generated stories. We can see that our judge matches that of (Chen et al., 2023) justifying its use in our experiments.

Judge	Pearson	Spearman
Ours	51.6	52.2
Chen et al. (2023)	53.5	50.8

Table 2: Comparison of our judge to that of (Chen et al., 2023) using human annotations on OpenMEVA-ROC (Guan et al., 2021). Our judge matches their performance justifying its use in this paper.

Political lean Similarly, we used LLM-as-a-judge to evaluate the political lean of texts. We asked LLaMa-3.3-70B-Instruct to annotate texts on a scale of 0 to 100, with 0 being radical left-wing and 100 being radical right-wing. Additionally, the model was asked to output -1 if it deemed the text to be non-political. We used the following prompt:

Political Lean evaluation prompt

```
Score the following post on a continual scale from 0
(left-wing) to 100 (right-wing) where a score of 0 means
"The post reflects radical left-wing ideology" and a
score of 100 means "The post reflects radical right-wing
ideology", and a score of 50 means "The post reflects
a neutral ideology". Reply with -1 if the post is not
political. Reply ONLY with the integer score (0-100 or
-1). DO NOT reply with text. Here is the text: <text>
```

B.3.1 Data properties selection

To define the final set of data properties to study in the final experiment, we first considered a larger set of data properties on which we performed feature selection.

Here we outline all the metrics considered.

- Quality: described in Appendix B.3.
- Semantic Diversity: We compute several semantic diversity metrics as pairwise cosine diversity in the embeddings of the `stella_en_1.5B_v5` model (Zhang et al., 2024a). Cosine diversity computes the pairwise diversity between all data points, and k-nn Cosine diversity computes the pairwise diversity for the nearest n neighbors (we use this metric with n=50 and n=1000).
- Lexial diversity: Self-BLEU (Zhu et al., 2018) is a metric that computes the average BLEU(Papineni et al., 2002) score for each text, with all other text taken as references.
- Word Entropy: computes the entropy using the word frequencies in the given texts
- Type Token Ratio (TTR) (Johnson, 1944): calculates the number of unique words (types) divided by the number of total word in the first 200 characters of each text.
- Text Length: average number of characters in each text.
- Positivity: uses the SentimentIntensityAnalyzer tool from NLTK (Hardeniya et al., 2016))(Apache) to assign a sentiment score for the text, ranging from -1.0 (highly negative) to 1.0 (highly positive).
- Toxicity: quantifies the presence of rude, disrespectful, or unreasonable language, using a probability score that ranges from 0.0 (benign and non-toxic) to 1.0 (highly likely to be toxic), as estimated by the classifier introduced in (Hanu and Unitary team, 2020).

- KL-Entropy: fits a 2D UMAP on the `stella_en_1.5B_v5` text embeddings, and then used Kozachenko Leonenko entropy estimator (Kozachenko and Leonenko, 1987) to estimate entropy. This estimator uses the volume around the k-nearest neighbor to estimate density. We create 2 kl_entropy metics, one with k=50 and one with k=1000.

- Gaussianity: uses the same UMAP representations as KL-entropy. In this space, it fits a 2D Gaussian distribution. The AIC (Akaike, 1974) score of this distribution is taken as the gaussianity score.

Figure 7 shows Variance Inflation Factor (VIF) scores before and after predictor variable selection. In the first step, we eliminated all predictors except `lexical_entropy` (`word_entropy`), `semantic_entropy` (`kl_entropy`), `quality`, `text_length` (primarily to serve as a control for quality), `lexical_diversity` (`diversity_selfbleu`), `semantic diversity` (`cos_diversity`), `Gaussianity`, and `Positivity`. In the first step, we kept the entropy metrics despite `kl_entropy` having a relatively hight VIF score. With those predictors, we conduct a pilot regression analyses (alike those in section 4.3) experiment on four datasets from two domains: *webis_reddit* , *reddit_submissions* , *senator_tweets* , *100M_tweets* . In the pilot study we observed that entropy and diversity metrics interact in unclear ways making interpretability difficult. Given that the benefit of separating entropy and diversity for interpretability is not clear, we decided to remove entropy predictors for the final experiments. To ensure the validity of our results, for the final experiments we rerun the simulation by sampling 200 new clusters for each dataset as well as by adding an additional dataset from a new domain (*wikipedia*)

B.4 Clustering

To obtain data to be used for regression analysis, we create a number of subsets from each of the datasets. This is done by the following procedure. A 2D UMAP is fit on the 90k text embedded with `stella_en_1.5B_v5` model (Zhang et al., 2024a). A series of clustering methods (dbscan, hdbscan, gmm, k-means) with different hyperparameters are done separately, each annotating the 90k samples. Then the rest of the dataset is annotated by k-nn, with k=1. This is done in two ways, with and without excluding the *noise cluster* for the k-nn classifier. This gives us a total of 120 different clusterings of the dataset. Then for each clustering 10 clusters are taken, if there are not enough clusters over the size of 60k. The remaining clusters are constructed by merging smaller clusters, either by uniformly sampling which cluster to merge or by iteratively merging the cluster that is the furthest away from the currently merged cluster. As this results in a large number of clusters (e.g. 1088 for *webis_reddit*), we then obtain the final cluster set by subsampling 200 clusters.

To clarify, from the 200 clusters created for each dataset, some are created in a straight forward way (e.g. with k-nn) which will create the nice narrow clusters as the reviewer describes. However, as discussed in this section, some clusters are created in a slightly more complex way, precisely to address this issue. Some of the used clustering methods create very small clusters (miniclusters) which are then merged to create bigger final clusters to be used in the experiments. This is sometimes done by randomly selecting the miniclusters to merge, and sometimes by iteratively adding the farthest away minicluster from the already merged miniclusters. Therefore, the final set of 200 clusters will be quite diverse.

B.5 Fine-tuning procedure

We use the Unsloth library (Daniel Han and team, 2023)(Apache license) to train model suing LoRA (Hu et al., 2021). The hyperparameters used are the default ones given by Unsloth, i.e. rank = 16, alpha = 16 batch size = 16, leaning rate = 2e-4, we use a linear schedules with 5 steps of warm-up. For generation with use a temperature of 1.5 with min_p (Nguyen et al., 2024) of 0.2.

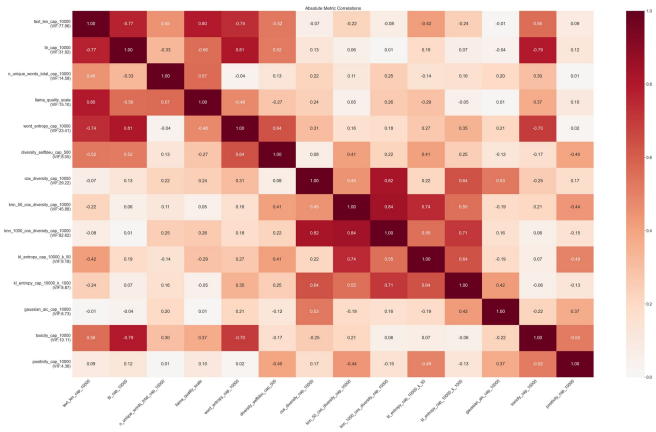
B.6 Iterative chain pseudocode

Figure 8 shows the pseudocode describing the iterative chain experiment used. Each iteration a fresh base model is selected and finetuned.

B.7 Discussion on corrections for multiple comparisons

We report uncorrected p-values in our regression analyses as our goal is not to test specific hypotheses about individual coefficients, but rather to identify general patterns of robust influence across datasets and conditions. On a related note, variable selection was conducted prior to these analyses and applied to newly collected data (last sentence in

VIF scores before variable selection



VIF scores after variable selection

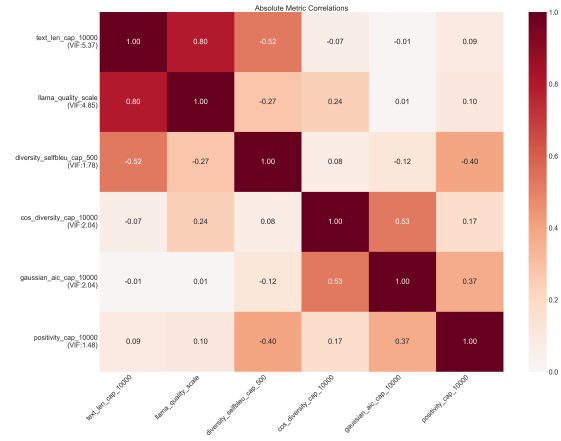


Figure 7: **Variable Inflation Factor (VIF) scores before (left) and after (right) predictor variable selection**

```
function iterative_chain(human_data, ai_ratio):
  base_models = [llama-3.2-1B, Qwen2.5-1.5B, SmolLM-1.7B, Falcon3-1B-Base]
  accumulated_data_pool = []
  for i in 20:
    iteration_base_model = sample(pretrained_models)

    if i == 0:
      training_set = sample(human_data, 8000)
    else:
      training_set = sample(accumulated_data_pool, 4000)

    iteration_ft_model = iteration_base_model.train_with_lora(training_set)

    iteration_new_data = iteration_ft_model.generate(4000*ai_ratio)
    iteration_human_data = human_dataset.sample(4000*(1-ai_ratio))

    accumulated_data_pool.add(iteration_new_data)
    accumulated_data_pool.add(iteration_human_data)
```

Figure 8: **Iterative chain pseudocode**

Appendix B.3.1). We therefore believe that the risk of multiple testing bias is minimal.

In other words, we address the issue of Type I errors in the way we interpret our results and make conclusions (i.e. in a way that is robust to potential Type I errors): the fact that we consistently find the same predictors to be significant across different experiments gives a high confidence that those are not statistical artifacts.

C Additional results

C.1 Increasing the number of models per generation

In the main text, we adopted the same experimental design as previous studies, where at each generation a single model is fine-tuned and used to generate new data. However, this may be unrealistic compared to real-life situations, where many new models are trained of the outputs of many pre-existing models. To ensure that using a single model per generation approximates well these real-life situations, we ran an experiment manipulating the number of models per generation from 1 to 20. In Figure 9), we observe that increasing the number of models per generation does not qualitatively change the conclusion about the effect of synthetic data ratio on distribution shifts with respect to quality and diversity. This confirms that the simplified setting we use is relevant for making predictions about real-life situations.

As a note, this pilot experiment was conducted with a slightly different quality metric, which ranks quality as either 0, 1 or 2. Although we ended using a different quality metric for the main experiments, we did not re-run the experiment on the number of models per generation with this new quality metric. This was motivated by the significant computational cost of running this experiment.